

Myntra Wishlist Discovery Engine – Technical Deep Dive

How 11,055 Myntra Reviews Become a Scored, Grounded Opportunity Backlog for Wishlist-to-Purchase Conversion

This document explains what the engine is, how it works end to end, the reasoning behind each design decision, and the technical constraints that shaped it — including the ones that broke and had to be fixed mid-build. It is written so an evaluator can verify the engineering, not just the narrative.

1 · The Problem This Engine Solves

Myntra's wishlist is a huge, mostly-passive save pile. Before proposing anything to fix wishlist-to-purchase conversion, a PM needs to know *why* saved items don't convert — at scale, from real customer language, not anecdote. The raw material exists (public reviews across Google Play, App Store, YouTube, MouthShut, and X) but is unusable by hand, and it has a real limitation baked in: app-store reviews were never designed to explain wishlist behavior. Most reviewers are commenting on delivery, refunds, and product quality, not on why they saved something instead of buying it. This engine is built around that constraint rather than around it:

- **Volume:** 11,055 reviews across 4 live channels — no product team reads that manually.
- **Noise:** the vast majority of the corpus is generic delivery/refund/quality complaints, not wishlist narrative — only a small slice is directly on-topic, and the engine says so out loud rather than inflating it.
- **Trust:** any claim a PM acts on needs to be checkable against the original review, not a paraphrase.

The engine turns that pile into a 14-area opportunity taxonomy scored on Reach × Impact × Confidence, a 5-segment breakdown of *why* people wishlist at all, and a grounded chatbot — every score and every quote traceable back to a specific review_id.

2 · Why Rule-Tag + Scored-Sample + LLM-Synthesize (and Not the Alternatives)

Approach	Why Not
Read reviews manually / spreadsheet tagging	Weeks of effort; subjective; doesn't scale; zero reproducibility.
Dump all 11,055 reviews into one giant LLM prompt	Impossible on the free tier in use — ~11K reviews is roughly 600K-700K input tokens, vastly over Groq's real 8,000 tokens/minute and 200,000 tokens/day caps for this account (verified live against the Groq console, not a guess). Also unauditable.
Pure keyword/regex search only	Cheap and runs on 100% of the corpus, but noisy on its own — see §6: regex-only precision against LLM judgment ranges from 24% to 59% agreement depending on the area. It also can't write a synthesis or judge severity.
Rule-tag + Scored Sample + LLM-Synthesize (Chosen)	Regex tags 100% of the corpus for free (Reach); a small, deliberately-scored sample per taxonomy area gets an LLM severity read and synthesis (Impact); the gap between the two becomes an honest Confidence number instead of a hidden assumption.

Core principle — grounding: the LLM is treated as untrusted until its output is tied back to real review_ids present in the corpus. Every synthesis quote is checked for verbatim substring match against the original review, not just plausibility. This is what prevents hallucinated opportunities and makes the Scorecard safe for a PM to act on.

3 · End-to-End Workflow & Two Processing Pathways

The system runs two pathways off the same corpus and vector index: an **offline batch pathway** that produces the Opportunity Scorecard and Wishlist Segments once per run, and an **online RAG pathway** that answers ad-hoc questions live in the dashboard.

Pathway A — Offline Opportunity Scoring

Stage	What Happens	Tech / Decision
Ingest	Pull public reviews across up to 5 channels (Google Play, App Store, YouTube, MouthShut, X), 10-week rolling lookback, merged incrementally.	google-play-scraper, custom App Store fetcher, YouTube Data API, Playwright+BeautifulSoup for MouthShut, X fetcher
Normalize	Dedupe near-identical reviews; filter short noise (<6 words); unify schema across channels.	Deterministic Python pipeline → data/processed/normalized_reviews.json
Embed & Index	Convert each review to a 384-dim meaning vector; store in Chroma keyed by review_id, tagged with rule-based taxonomy metadata for scoped retrieval.	Local sentence-transformers all-MiniLM-L6-v2 + Chroma DB (collection myntra_reviews)
Rule-Tag (\$0)	Regex-match all 11,055 reviews against all 14 taxonomy areas — free, runs on 100% of the corpus regardless of which areas get LLM synthesis this run.	src/analysis/taxonomy.py::classify_rule_based
Score-Sample	Per area, rank matched reviews by keyword density + length quality + rating friction + helpfulness + a novelty bonus for not-yet-LLM-tagged reviews; take the top 40.	src/analysis/sampler.py (OPPORTUNITY_SAMPLE_CAP_PER_AREA=40)
LLM-Synthesize	Batch the sample (20/call) to Groq for a relevance + severity judgment and a grounded synthesis paragraph; results cached by content-hash + prompt-version so repeat runs don't re-spend tokens.	Groq openai/gpt-oss-120b, temperature=0.2, JSON mode; src/analysis/tag_cache.py
Score	Blend free Reach stats with LLM Impact/Confidence into one comparable signal_score per area; merges into existing results rather than overwriting untouched areas.	src/analysis/scoring.py → opportunity_scores.json
Validate	Enforce PII scrubbing, 200-word synthesis cap, and verbatim quote-in-corpus checks before anything is written to disk.	src/analysis/validators.py + opportunity_run_metadata.json

Pathway B — Online Live Chat (RAG)

Question → out-of-scope gate (regex refusal for anything not about Myntra/reviews) → Chroma retrieve (top_k=8 from a fetch pool of 40, MMR re-ranked with $\lambda=0.7$ for relevance/diversity balance) → similarity threshold gate (0.30) → shared daily budget gate (see §6) → Groq generate → citation validation → **fallback to a free retrieval-only answer on any failure at any step**, never a hard error shown to the user. The same pipeline can be scoped to a single opportunity area (Deep-Dive screen) by filtering on that area's tag metadata.

4 · What "Rule-Tagging," "Scored Sampling," and "RAG" Actually Mean (Plain Version)

Rule-tagging is a free, deterministic pass: does this review's text contain any of a taxonomy area's keyword phrases (e.g. "out of stock", "not sure which one to buy")? It runs on every review, costs nothing, and gives an honest reach number — but it's a proxy, not a judgment; a review can contain a keyword and still not really be about that topic.

Scored sampling decides which of the (possibly thousands of) rule-matched reviews are worth an expensive LLM call: longer, more critical, more helpful, and not-yet-reviewed reviews score higher, so each LLM call is spent where it teaches the most.

RAG (Retrieval-Augmented Generation) is the live-chat pattern: embed the question, retrieve the handful of reviews most similar in meaning, and generate an answer using only those retrieved reviews. The LLM never answers from general internet

knowledge — it answers strictly from retrieved evidence, which is what makes every claim citable.

5 · Why Local Embeddings, Groq for Generation — and a Real Mid-Build Model Swap

The architecture pairs a local embedding model with a hosted generation API, and that generation model changed mid-build for a documented reason worth surfacing rather than hiding:

Task	Engine Choice	Why
Embedding 11,055 reviews + each query	Local MiniLM (384-dim)	Free, 100% offline, zero API token budget consumed, fast on CPU.
Opportunity synthesis + segmentation + chat answers	Groq openai/gpt-oss-120b	Strong JSON-mode reasoning at interactive Groq LPU speed.

The model swap, in order: the pipeline originally targeted **llama-3.3-70b-versatile**, which Groq removed from its lineup entirely (confirmed live via a 404 model_not_found). It was replaced with **openai/gpt-oss-20b**, which then hit its real daily token ceiling mid-run and started returning escalating 429s (264s → 432s Retry-After). The fix was switching to **openai/gpt-oss-120b** — a separate, independent quota bucket on the same account, confirmed with a live test call before committing to it. Both incidents are why the client now caps any Retry-After wait at 60 seconds and fails fast and visibly instead of silently blocking for 15+ minutes, which happened once for real during development.

6 · Why 40 Reviews Per Area, Not the Full Corpus (The Token-Limit Math)

This is the binding engineering constraint, and the numbers below are the real ones verified against the Groq console for this account's **openai/gpt-oss-120b** and **openai/gpt-oss-20b** models (both show identical limits) — not a placeholder guess. An earlier internal budget config used 500,000 tokens/day as a guess; the real console value is 200,000, and being 2.5× too generous is exactly what let a run blow past Groq's real server-side ceiling before the tracker caught it.

The Rate Limit Numbers (verified, this Groq account):

- 8,000 tokens per minute (TPM), throttled to 7,000 with local headroom
- 200,000 tokens per day (TPD)
- 30 requests per minute (RPM), 1,000 requests/day (RPD)

The math: a typical review is ~50–60 tokens. 11,055 reviews ≈ 570,000–660,000 input tokens. That is:

- ≈ **80× over the 8,000 tokens-per-minute ceiling**, and
- ≈ **3× over the 200,000 tokens-per-day cap** — on a single pass, before even accounting for output tokens.

The fix — per-area scored sampling + a persistent tag cache: `sampler.py` caps each of the 14 taxonomy areas at 40 reviews (OPPORTUNITY_SAMPLE_CAP_PER_AREA), and a content-hash + prompt-version keyed cache (`llm_tag_cache.json`, currently ~270KB) means a review already LLM-tagged for an area is never re-spent on in a later run — the "novelty bonus" in the sampler actively steers new runs toward uncovered reviews. The day this document was generated, a full run cost only **40 Groq calls and ~131,000 tokens** — well inside budget. On top of that, the shared budget tracker (`src/analysis/budget.py`) reserves **25% of the daily ceiling for live chat** at all times, so a background batch run can never starve a user asking a question in the dashboard.

7 · The Trust Layer — How Hallucination Is Prevented

LLMs can invent plausible-sounding claims. For a PM making roadmap bets, that's dangerous. The engine defends against it with a deterministic validation layer that runs on every synthesis and every chat answer:

Check	Rule Enforced
Provenance	Every synthesis quote must appear verbatim (whitespace-normalized substring match) inside the actual sampled review, not just be plausible; every chat answer must cite a review_id that was genuinely retrieved for that question.

Structural Integrity	Synthesis capped at 200 words; at least 1 supporting quote required; malformed JSON from the model triggers a retry, not a silent skip.
Privacy	Emails and phone numbers are regex-scrubbed from synthesis and chat answers before display (citation tokens are excluded from the scrub so review_ids survive).
Scope Enforcement	Out-of-scope questions are refused via regex gate before ever reaching retrieval or the LLM; low-similarity questions (<0.30) fall back to a free retrieval-only answer instead of guessing.

8 · The Opportunity Scorecard — Real Results Across 14 Taxonomy Areas

Every area below is scored on the same three axes: **Reach** (% of the 11,055-review corpus a free regex match hit), **Impact** (blended rating-drop + LLM-assessed severity), and **Confidence** (how much of the matched population the LLM actually reviewed, and how often it agreed the match was genuine). The ranked table below is the live opportunity_scores.json output, not illustrative numbers.

Opportunity Area	Reach	Impact	Confidence	Signal
Trust / Review Credibility Gap	7.6% (841)	67.8	35.1	72.5
Quality / Material Doubt	20.7% (2,286)	51.3	40.3	68.0
External Validation-Seeking	2.4% (267)	40.6	40.5	64.3
Stock / Size / Color Availability Friction	0.1% (16)	72.0	96.3	51.6
Delivery / COD / Payment Friction	0.3% (37)	73.6	95.5	50.3
Size & Fit Uncertainty	0.5% (50)	65.4	75.3	49.2
Return / Exchange Anxiety	4.8% (533)	84.7	32.3	47.0
Comparison Paralysis	3.5% (387)	40.0	47.2	32.6
Wishlist as Price-Drop Watch	0.05% (6)	49.3	48.6	30.2

Reach and Confidence deliberately pull in opposite directions here: **Quality/Material Doubt** has the 2nd-highest reach (2,286 matches) but the LLM has only sampled 2.4% of it, so Confidence stays modest (40.3) — it's a high-volume area still under-verified. **Stock/Size/Color Friction** and **Delivery/COD/Payment Friction** sit at the opposite extreme: small enough (16 and 37 matches) that the LLM has reviewed nearly all of them, so Confidence is over 95 even though Reach is tiny. Four remaining MVP areas — Styling & Occasion Uncertainty, Wishlist as Bookmark/No Intent, General Price Hesitation, and the two Wishlist-as-Gift/Inspiration areas — are in the full taxonomy and scored artifact but omitted from this table for space; two of those (Styling & Occasion Uncertainty, Wishlist Bookmark-No-Intent) currently show 0 matched reviews after their keyword sets were deliberately tightened mid-build to kill false positives — see §10.

9 · Wishlist Usage Segments — Why People Wishlist At All

A separate, single-label LLM pass classifies wishlist-mentioning reviews into one of 5 MECE behavioral segments (plus "unclear"). The candidate pool is deliberately narrow — only reviews that literally say "wishlist" or match one of 4 wishlist-motivation taxonomy areas — because most of the corpus simply isn't commenting on wishlist behavior at all:

Segment	Count	% of Classified
Unclear / Insufficient Signal	52	89.7%
Price-Drop Watch	3	5.2%
Genuine Purchase Intent	2	3.4%
Bookmark, Not Intent	1	1.7%
Gift List / Inspiration Board	0 / 0	0.0%

Out of a 73-review candidate pool, 58 were classified — and 89.7% of those landed in "Unclear." This is reported as-is rather than smoothed over: it's the honest ceiling of what public app-store reviews can tell you about wishlist motivation. Most of this

corpus is star-rating reviews about delivery, refunds, and quality — not reflective narratives about why someone saved an item instead of buying it.

10 · Honest Limitations

- **Inferred, not measured:** this engine has no access to Myntra's real wishlist telemetry — every behavioral claim is inferred from review text, not verified against actual user actions.
- **High-reach areas can still be low-confidence:** Quality/Material Doubt (20.7% reach) has only been 2.4% LLM-sampled — treat its synthesis as directional, not exhaustive, until more of it is reviewed.
- **Regex pre-filter is noisy by design:** rule/LLM agreement ranges from 24.1% (Trust) to 59.3% (Quality) across areas — the free reach number is a proxy for "worth investigating," not ground truth on its own.
- **Two MVP taxonomy areas currently show zero reach:** Styling & Occasion Uncertainty and Wishlist-as-Bookmark keyword sets were deliberately tightened mid-build after they were matching generic praise and unrelated complaints (documented inline in taxonomy.py); they now return 0 true matches, which likely means the real signal needs an LLM-only classification pass rather than a keyword pre-filter, not that the behavior doesn't exist.
- **Corpus is skewed positive:** 64.2% of the 11,055 reviews are 5★ and only 23.5% are 1★ — reach percentages are relative to a mostly-satisfied corpus, not a curated complaint set.
- **Reddit was evaluated and removed:** both direct PRAW access and two Apify no-login fallbacks proved unreliable as a 6th channel and were dropped entirely (see git history). X/Twitter ingestion code exists but returned 0 reviews in the current corpus snapshot.
- **Ephemeral vector store:** on an ephemeral-disk deploy target, the Chroma index would need to rebuild on cold start; this is not yet an issue since the app currently runs locally, not on a hosted platform.

11 · Tech Stack Summary

Layer	Technology Choice
Ingestion	google-play-scraper, custom App Store fetcher, YouTube Data API, Playwright + BeautifulSoup for MouthShut (shuffled page-order pagination to avoid a confirmed silent-repeat bug), X fetcher
Embeddings	sentence-transformers all-MiniLM-L6-v2 (local, 384-dim)
Vector Store	Chroma DB, collection "myntra_reviews", file-persisted, keyed by review_id with taxonomy-tag metadata for scoped retrieval
Generation	Groq openai/gpt-oss-120b (JSON mode, temperature 0.2 batch / RAG generator), shared daily budget tracker with a 25% live-chat reservation
Dashboard	Streamlit — 5 screens: Opportunity Scorecard, Live Chat, Review Explorer, Opportunity Deep-Dive, Wishlist Segments
Deployment	Local Streamlit run (not yet pushed to a hosted platform as of this document)